



ENGINEERING · HIRING

Voice AI Pipeline *Take-Home*.

A focused technical exercise for senior full-stack AI engineering candidates. Build a single-call demo of a Retell / Vapi / Bland-style voice agent, with the real signal living in the alignment between what was said and what was heard.

ROLE	TIME	STACK	SUBMISSION
Sr. Full-Stack AI Developer	8–10 hours Window: up to 5 days	Python · TypeScript React frontend	Repo + Demo 4–5 min recording

01 Context

We build voice AI agents in the same category as Retell, Vapi, and Bland. This exercise tests the part of that stack that's actually hard — the real-time audio loop and the transcript-alignment problem — without making you build a full platform.



REFERENCE REPOSITORY · FORK, IGNORE, OR READ

github.com/twilio-samples/speech-assistant-openai-realtime-api-node

02 What you're building

A single-call demo. Three pieces, each owning a clear slice of the stack:

PYTHON

Real-time loop

Ingests audio frames, runs STT → LLM → TTS, handles barge-in, writes aligned transcripts.

TYPESCRIPT

Transcript API

Reads the transcripts and exposes two endpoints — `GET /calls` and `GET /calls/:id`.

REACT FRONTEND

Transcript UI

One page that hits the API and renders the transcript chat-bubble style, with visible interruption markers.

IMPORTANT

You may — and should — mock every external provider. No real Twilio, no real Deepgram, no real ElevenLabs. The *interfaces* and the *timing logic* are what we're grading, not the integrations.

Out of scope — real telephony, SIP setup, production deployment, auth, multi-tenancy, high concurrency, UI polish. If you find yourself working on these, stop and refocus.

The pages that follow break down each piece in turn, with explicit acceptance criteria. The transcript-alignment work in **Section 3.2** is the part we weight hardest in scoring — read it before you start writing code.

03 Requirements

Five sub-sections. The first establishes the Python-side real-time loop. The second — aligned transcripts under interruption — is the part we weight hardest. The third covers latency awareness. The fourth describes a thin TypeScript layer and minimal UI. The fifth specifies how the agent should sound.

3.1 REAL-TIME LOOP · PYTHON

You can use the OpenAI Realtime API as a single-provider shortcut, or wire up a chained pipeline (e.g. Deepgram → LLM → ElevenLabs). Either is fine — articulate the trade-off in your README.

A Python service (FastAPI is the obvious choice) exposing a WebSocket that:

- Receives audio frames from a fake client — e.g. a script that streams a wav file in real time.
- Pipes them through a mock STT that emits interim and final transcripts on a timer.
- Calls an LLM and streams the response. Real model (cheapest, fine) or a canned-response stub that streams tokens with a small delay — either is acceptable.
- Pipes LLM output through a mock TTS that produces audio frames back over the same WebSocket.
- **Handles barge-in:** when the user starts speaking mid-utterance, immediately stop sending TTS audio and cancel the in-flight generation.

Audio format is your choice — μ -law or PCM. Document what you picked and why.

3.2 ALIGNED TRANSCRIPT

Per call, persist a JSON record covering every utterance in both directions, with millisecond-level timing. The exact schema is yours to design, but it must capture:

- Speaker — user or agent.
- The exact text that was **spoken or played**, not what was generated.
- Start and end times as call-time offsets, in milliseconds.
- An explicit signal when an agent utterance was truncated by interruption.

A workable shape:

```
{
  "call_id": "abc123",
  "utterances": [
    { "speaker": "user", "text": "I need help with my bill",
      "start_ms": 1200, "end_ms": 3400 },
    { "speaker": "agent", "text": "Sure, I can help with your bill—",
      "start_ms": 3600, "end_ms": 5200, "interrupted": true },
    { "speaker": "user", "text": "actually it's about my plan",
      "start_ms": 5100, "end_ms": 6800 }
  ]
}
```

Note the 100ms overlap on barge-in (user-2 starts at 5100ms while agent ends at 5200ms). That's the realistic detection latency between the user speaking and the system cutting agent audio — not a typo.

THE SIGNAL WE WEIGHT HARDEST

If the agent was interrupted after "Sure, I can help with your bill—", the transcript must show exactly that. Not the full intended sentence the model was about to generate. Easy to fake on the surface, hard to get right under interruption.

3.3 LATENCY AWARENESS

Voice-to-voice latency under **~800ms** is the industry target — anything slower and the conversation feels broken. You won't hit that under mocks, and we don't expect you to. We *do* expect: instrumentation at each stage (STT first-final, LLM first-token, TTS first-byte, voice-to-voice total) with numbers reported honestly in your README, and a streaming-everywhere architecture with no buffering between stages. The latency-friendly choices are what we're actually grading here.

3.4 API AND UI · TYPESCRIPT

Pick whichever stack you prefer — Next.js, NestJS, Express, Fastify, React, or anything else in the TS / JS ecosystem. Two endpoints required:

- `GET /calls` — list call IDs.
- `GET /calls/:id` — return the aligned transcript.

Storage: SQLite or a JSON file on disk. No auth, no migrations. One page (React, Next, or your framework of choice) that renders the transcript with both speakers, timestamps, and a visible interruption marker when an agent utterance was cut short. We're not grading polish — we're grading whether the interruption is visible to the reader.

3.5 VOICE-OUTPUT RULES IN THE SYSTEM PROMPT

Hardcode a system prompt that includes spoken-text formatting discipline:

- Plain spoken text. No markdown, bullets, or emoji.
- One to two sentences per turn.
- Numbers and acronyms spelled out: `"$130,000"` → *"one hundred thirty thousand dollars"*,
`"API"` → *"A P I"*.

04 Evaluation

01 Barge-in correctness.

Does the transcript actually reflect what the caller would have heard?

02 Architecture.

Where did you draw the Python / TS line, and why? What happens when the WebSocket dies mid-call?

03 Code quality.

Types, error handling, module boundaries. No monolithic 600-line files in either service.

04 Domain awareness.

Your README should make it clear you know *why* this stack looks the way it does.

05 Stretch — *only if core is done. Pick one, max.*

- Wire in **real STT or real TTS** through your mock interface, to prove the interface holds up.
- Add a small **RAG step** — one hardcoded document, top-k retrieval, injected into the prompt. In-memory vector store is fine.
- Add one **function-calling tool** the agent can invoke mid-call. Something like `lookup_customer({phone_number})` — it returns a customer record the LLM weaves into the next utterance, exercising the full round-trip.

06 Deliverables

01 Repository

`docker compose up` — or two to three commands — to run end-to-end.

02 README

How to run it · architecture sketch · trade-offs you made · what you would build next with another day.

03 Demo recording 4–5 minutes

Loom or screen capture. Start a simulated call, interrupt the agent mid-sentence, open the UI, show the transcript matches what actually happened.

07 Ground rules

SUBMISSION

Submit by **9 June 2026**. Upload your repository link and demo recording to the ATS link sent by our HR team, and email a copy to

`Careers@softwarefinder.com`.

AI TOOLING

Cursor, Copilot, Claude — all allowed and encouraged. We will ask you to walk through any non-trivial section in the follow-up call. If you can't explain it, it doesn't count.

AMBIGUITY

If something is under-specified, decide and document. Don't email asking for clarification on scope — making the right call *is* part of what we're evaluating.

SCOPE HONESTY

A small, working, well-explained submission beats a sprawling broken one. Every single time. Ship what you have with an honest README.